

Aprendizagem Aplicada à Segurança

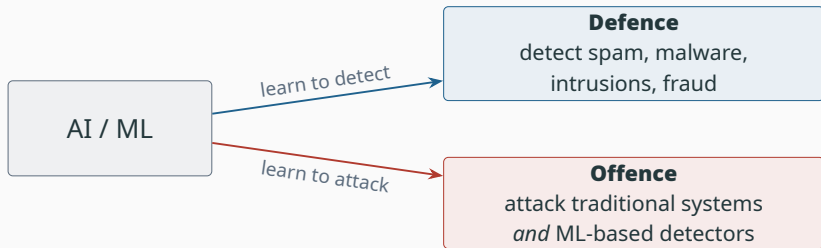
Class 1: Using AI/ML to Break Security

Mário Antunes

September 18, 2026

Universidade de Aveiro

AI for security: defence and offence

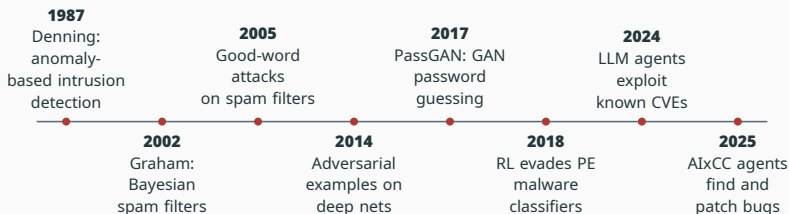


- **Class 1:** the toolbox (models, gradients, optimisation) and the map of what ML does for both sides.
- **Classes 2-4:** build a **spam detector**, then **break** it; later: anomaly and malware detectors.
- *Out of scope this year:* securing the AI pipeline (data provenance, model signing).

Rules of the game

- **Lab only.** Own models, own data, and targets we are authorised to test.
- **Dual use.** Every technique works for both sides; we ask *who benefits and at what cost*.
- **State the threat model** before any attack: goal, knowledge, capability, stage. *No threat model, no security claim*.
- **Measure the attack:** success rate **and** cost (edits, queries, time, compute).
- **Format:** each 3-hour class is **1 hour** of lecture and **2 hours** of guided lab (notebooks and practice guides).

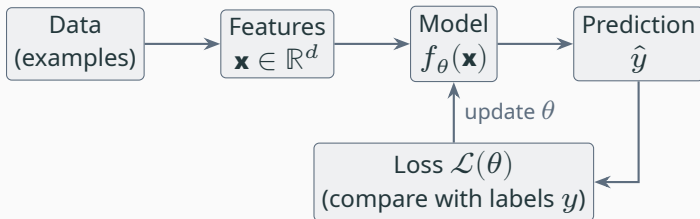
Security meets ML: a short timeline



- Defence: Denning (Denning 1987), Graham (Graham 2002). Attacks on detectors: (Lowd and Meek 2005; Szegedy et al. 2014; Anderson et al. 2018).
- Attacks with ML: PassGAN (Hitaj et al. 2019), exploiting CVEs (Fang et al. 2024), AIxCC (Zhang et al. 2026).

What is machine learning?

Traditional program: *rules + data $\rightarrow$ answers.* **Learning:** *data + answers $\rightarrow$ rules (a model).*



- A model is a **function with parameters θ** ; *training* chooses θ so that the loss is small on the data.
- Everything in this course, from Naive Bayes to LLMs, fits this picture.

Learning is optimisation, and so is attacking

Training

vary the **weights**

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta; \mathbf{x}, y)$$

descend the loss

Evasion

vary the **input**

$$\mathbf{x} \leftarrow \mathbf{x} + \varepsilon \operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\theta; \mathbf{x}, y))$$

ascend the loss

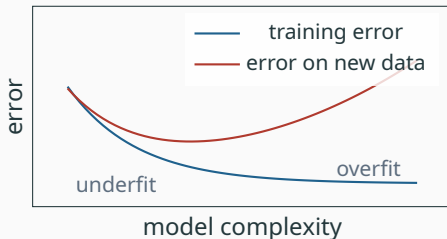
- **Same gradient, different variable.** The attacker uses the machinery that trains the model.
- No gradient (black-box API)? Use **derivative-free search**: particle swarms, genetic algorithms, greedy search.
- Details, maths and code: the **appendix** of this deck and notebooks/00-intro/ (02 to 05, then 11).

Learning paradigms, and who uses them

Paradigm	Learns from	Defence	Offence
Supervised	labelled examples	spam, malware classifiers	surrogate models of a victim
Unsupervised	unlabelled data	anomaly detection	profiling targets
Generative	data distribution	synthetic training data	phishing text, passwords
Reinforcement	reward	adaptive response	pentest agents, evasion
Self-supervised (LLMs)	text itself	alert triage	general-purpose agents

- Labels in security are **scarce, late and noisy**: practice leans on semi- and unsupervised methods and on pre-trained models.

Generalisation: the enemy of memorisation



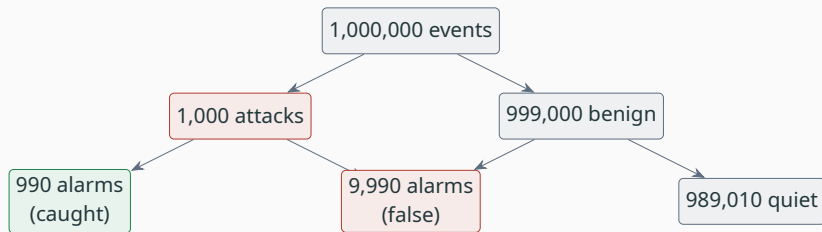
(Schematic.) A model that **memorises** the attacks it has seen is excellent in the lab and blind to the next variant.

Why generalisation matters in security

- **New variants:** attackers repack and rewrite; a detector must recognise what it has *not* seen.
- **Gaps between training points** are where a detector is unreliable: the attacker moves the input there.
- **Spurious correlations:** the model learns artefacts (data source, file size, HTML escapes), not the real cause (Arp et al. 2022).
- **Evaluation traps:** duplicates across train and test, random splits of time-ordered data, tuning on the test set.
- An evaluation with **no adaptive adversary** says nothing about security.

Metrics: the base-rate fallacy

A detector with **99%** detection rate and **1%** false-alarm rate.
Attacks are **1 in 1,000** events.



$$P(\text{attack} \mid \text{alarm}) = \frac{990}{990 + 9990} \approx 9\%: \textbf{91\% of the alarms are false}$$
 (Axelsson 2000).

Consequences of the base rate

- Real attacks hide in a **flood of alarms**: analysts stop looking at a detector that cries wolf.
- **Defender**: tune the operating point to analyst capacity; report precision and recall at the true prevalence.
- **Attacker**: noise is cover; more benign-looking junk means more alarms to ignore.
- **Rule for all our labs**: precision, recall, PR-AUC and MCC on a *leak-free* split, as mean $\pm$ spread over folds.

Where ML helps defenders

Problem	Data	Typical models	Where in this course
Spam, phishing	message text, URLs	Naive Bayes, linear, embeddings	classes 2-4
Malware	PE headers, API calls, bytes	trees, ensembles, nets	malware module
Intrusion, anomaly	flows, logs	isolation forest, autoencoders	anomaly module
Fraud, UEBA	transactions, user behaviour	boosting, one-class	anomaly module
Triage, threat intel	alerts, reports	LLMs, retrieval	discussion

- **Why learning:** it generalises to *variants*, adapts to drift and scales to millions of events per day.

Signatures and rules versus learning

	Signature / rule based	Learning based
Spam	keyword and blocklist rules	statistical text classifiers
Malware	hashes, byte patterns	classifiers on static and dynamic features
Intrusion	Snort/Suricata-style rules	anomaly detection on flows and logs
Strength	precise, explainable, cheap	generalises to variants
Weakness	blind to anything new; one changed byte evades it	needs data, harder to explain, statistical errors

- Defenders **combine both**: rules for the known, learning for the unknown.

Why security ML is different

Sommer and Paxson: detection in the lab is not detection on a real network (Sommer and Paxson 2010).

- **Costly errors:** a false negative is a breach, a false positive burns analyst time.
- **Semantic gap:** “anomalous” is not “malicious”; the operator needs *harmful*.
- **Few realistic, shareable, labelled datasets;** sites and threats differ and change.
- **Adversaries adapt** to the detector, so the data reacts to the defence.
- **Evaluation is hard:** one accuracy number hides all of the above.

Common pitfalls (Arp et al., 2022)

Pitfalls found in *every one* of 30 top-venue papers reviewed (Arp et al. 2022):

Pitfall	Example in security
Sampling bias, label noise	benign and malicious samples from different sources
Data snooping	tuning or selecting features with the test set
Spurious correlations	model keys on artefacts, not on behaviour
Wrong metrics	accuracy on balanced data, not PR at real prevalence
Temporal bias	random splits mix past and future

The offensive AI landscape

Task	Classic technique	With ML / AI
Passwords	dictionaries, rules (hashcat)	generative models of leaked passwords (Hitaj et al. 2019)
Phishing	templates, mass mailing	LLM-written, personalised, fluent text
Bug finding	fuzzers, manual audit	LLM-guided fuzzing, autonomous find-and-patch (Zhang et al. 2026)
Pentesting	scripts, scanners	LLM agents that plan and use tools (Deng et al. 2024)
Exploitation	manual	agents exploiting known CVEs (Fang et al. 2024)
Malware	packers, polymorphism	RL and generators that evade AV (Anderson et al. 2018)
CAPTCHA, biometrics	brute force, replay	CNNs solve them; deepfakes

Case 1: password guessing with generative models

A password is a *string with structure*; a language model assigns it a **probability**:

$$P(c_1 \dots c_n) = \prod_{i=1}^n P(c_i \mid c_1 \dots c_{i-1})$$

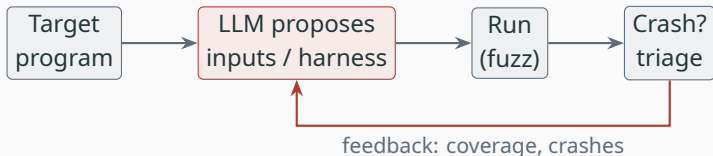


- Rules (p4ssw0rd!) are hand-written; a model **learns the habits** of real users from data (Hitaj et al. 2019).
- *Offline* attack: only a leaked hash and compute.

Case 2: social engineering at scale

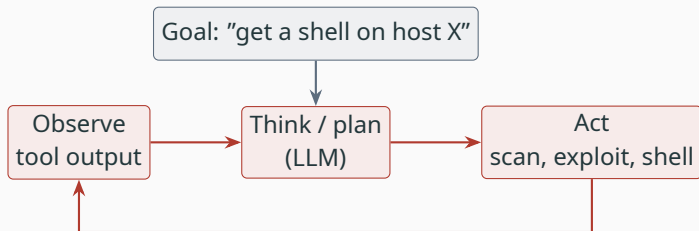
- Classical phishing has tell-tale signs: **generic greeting, typos, odd phrasing, one template** for a million victims.
- An LLM removes them: **fluent, personalised, multilingual** text produced from public information about the target, at almost zero marginal cost.
- Voice and video cloning add a second channel (fake calls from a “manager”).
- Content signals weaken; the spam filter of classes 2-4 will be attacked with exactly such texts (notebook 01-spam/07).

Case 3: finding and exploiting vulnerabilities



- In DARPA's **AIXCC** (2023-2025) seven finalist *cyber reasoning systems* used LLMs to **discover, confirm and patch** bugs in real open-source software (Zhang et al. 2026).
- LLM agents can exploit **known** (one-day) vulnerabilities when given the CVE description (Fang et al. 2024).

Case 4: LLM agents as penetration testers



- Systems such as PentestGPT (Deng et al. 2024) chain reconnaissance, exploitation and reporting.
- **Limits:** hallucinated findings, long-horizon planning, no proof of exploit: serious tools *demand a validated proof* before reporting.
- Trust in agent output rests on **validation:** a working proof of exploit, not a fluent report.

Why ML changes the economics of attacks

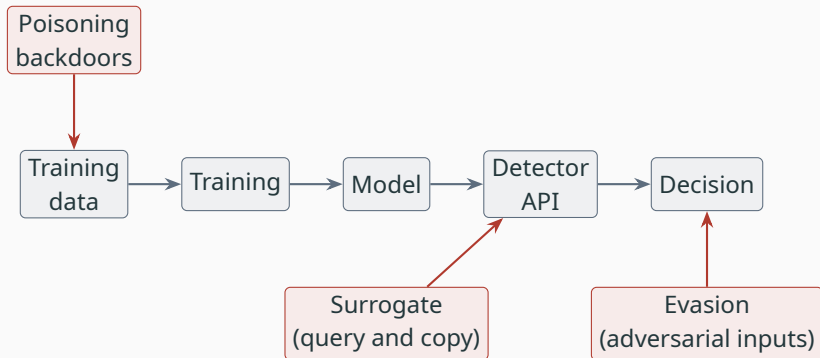
- **Scale:** thousands of personalised messages, millions of candidate passwords, continuous scanning.
- **Cost:** the skilled labour of a few experts is replaced by compute and prompts.
- **Adaptivity:** an attack that *learns from the defender's responses* (a score, a block, an error message).
- **Asymmetry:** the attacker needs one success; the defender must stop every attempt.
- Natural next question: what if the **defence is itself a model**? See the last section, and the spam lab.

Threat model: who attacks what, and when?

Axis	Options
Goal	evade (a malicious input is accepted), degrade (raise errors), learn the model
Capability	change test-time inputs; contribute training data or labels; query the detector
Knowledge	white-box (weights), grey-box, black-box (scores or labels only)
Stage	training time or inference time

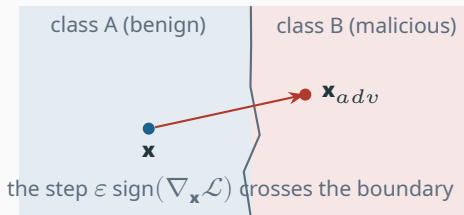
- **Targets in this course:** the spam filter (classes 2-4), the anomaly detector, the malware classifier.
- A detector is “robust” only against a *stated* adversary (Vassilev et al. 2025).

Where can a detector be attacked?



Evasion: the gradient points the way

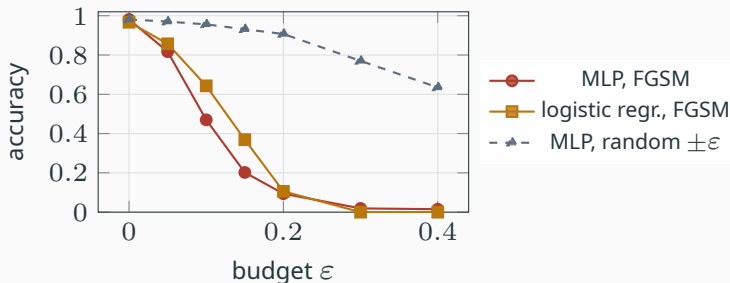
Fast Gradient Sign Method (Szegedy et al. 2014; Goodfellow et al. 2015): one step that increases the loss under an ℓ_∞ budget ε .



$$\mathbf{x}_{adv} = \mathbf{x} + \varepsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\theta, \mathbf{x}, y))$$

Evasion is not random noise

Small digit classifier (notebook 00-intro/11); same ℓ_∞ budget ε per pixel (pixels in $[0, 1]$).



In security, the perturbation must still work

- **Images:** any small change is allowed. **Security:** the input must keep its *function* (the malware runs, the spam is readable, the URL resolves).
- The attack becomes a **search over valid edits** ("problem space"), not a step in pixel space.

Detector	Valid edits	Reference
Spam text	append benign words, obfuscate spelling	(Lowd and Meek 2005)
PE malware	append bytes, add sections, repack	(Anderson et al. 2018)
Anomaly (flows, logs)	mimic normal statistics, stay below threshold	anomaly module

Black-box attacks: queries and transfer

- **Query attack:** ask the detector for a score on inputs you choose; lower it with derivative-free search (PSO, greedy, genetic).
- **Transfer:** train a **surrogate** on labels obtained by querying the victim, attack it with gradients; the examples *transfer* (Papernot et al. 2017; Tramèr et al. 2016).

Digits, $\varepsilon = 0.2$, 30 test images (notebook 00-intro/11):

Attack	Success rate	Cost
Random noise	0.07	none
FGSM (white-box)	0.90	1 gradient
PSO (black-box)	0.97	about 1,300 queries per image

Poisoning and backdoors

- **Poisoning:** contaminate the training data so the detector is wrong in a chosen way (Biggio et al. 2012).
- **Backdoor:** a hidden **trigger** flips the decision only when present; the model looks normal on clean data (Gu et al. 2019).
- **Realistic channels:** “Not spam” buttons, crowdsourced malware feeds and labels.
- The number of poisoned samples needed can stay **small**: about 250 documents backdoored LLMs from 600M to 13B parameters (Souly et al. 2025).
- Spam lab (01-spam/07): 50 to 100 relabelled messages (1-2% of the training set) cut the recall of a logistic-regression filter from 0.91 to 0.54 and 0.30.

Summary: attack, knowledge, cost

Attack	Knowledge needed	Attacker's cost	Where we practise it
White-box evasion (FGSM, PGD)	weights	1 backward pass	00-intro/11
Black-box evasion	scores	100s-1000s of queries	00-intro/11, 01-spam/07
Transfer	labels	queries to train a surrogate	01-spam/07
Good-word attack	weights or scores	a few appended words	01-spam/07
Poisoning, backdoors	write access to data	tens of samples	01-spam/07

Dual use: one technique, two sides

spam classifier	text models	LLM phishing writer
password-strength meter	sequence models	password guesser
static analysis, fuzzing	code models, LLMs	exploit finding
malware classifier	gradients, search	evasion of the classifier

- The same model families sit on both sides; what differs is the **objective**, the **error costs** and **who adapts to whom**.

The labs of class 1

Notebook (notebooks/00-intro/)	Topic
00, 01, 02	probability, linear algebra, calculus (gradients, integration)
03, 04, 05	PSO, gradient descent, linear regression: <i>the optimisers</i>
06	classifier zoo and decision boundaries
07-10	learning as compression, evaluation, pitfalls, taxonomies
11	attack lab: FGSM, PGD, transfer, PSO black-box

Core path for the 2 hours: 02 $\rightarrow$ 04 $\rightarrow$ 11. The others are self-study; 00 and 01 are the maths refresher.

Take-aways

- ML works for **both sides**: defenders detect (spam, malware, intrusions); attackers guess, phish, find bugs, and **attack the detectors**.
- **Learning is optimisation, and so is attacking**: gradients when available, derivative-free search when not.
- Security ML is **not** ordinary ML: rare positives, costly errors, drift, adaptive opponents. Judge it by **precision at the real base rate** and by **robustness to a stated adversary**, not by accuracy.
- **Next class**: we build a spam detector, from the simplest Naive Bayes to small-language-model embeddings, then break it.

Training is optimisation

- **Optimisation** finds the best solution of a problem: the **minimum** (or maximum) of a function.
- Training a model picks the parameters θ that minimise a **loss** (cost) over the data:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i)$$

Task	Model	Loss ℓ
Regression	$\hat{y} = mx + b$	squared error $(y - \hat{y})^2$
Classification	$\hat{p} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$	log-loss (cross-entropy)

Two families of optimisers

Gradient-based

needs $\nabla \mathcal{L}$

fast on smooth, convex losses

gradient descent, Newton,
L-BFGS, Adam

Derivative-free (blind)

needs only $\mathcal{L}(\theta)$

any function, black boxes, noise

PSO, genetic algorithms,
random search

- Choose by what you can **observe**: the gradient (white-box) or only the value (black-box, a *query*).

Gradient descent in one dimension

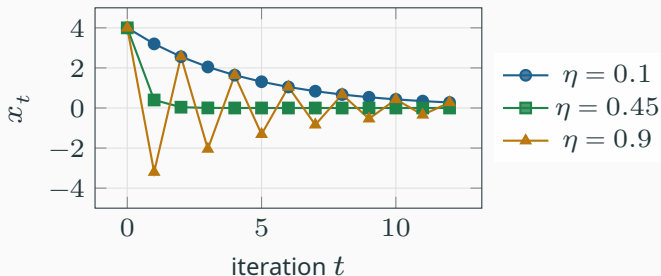
Imagine descending a foggy mountain: feel the slope, step downhill, repeat.

$$x_{t+1} = x_t - \eta f'(x_t)$$

- $f'(x_t)$ is the slope at the current point; η is the **learning rate**.
- **Too small:** very slow. **Too large:** overshoots, oscillates or diverges.
- For $f(x) = x^2$: $x_{t+1} = (1 - 2\eta) x_t$, so it converges only if $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$.

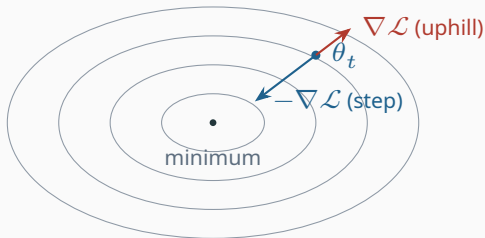
The learning rate decides everything

$f(x) = x^2$, start at $x_0 = 4$: $x_t = (1 - 2\eta)^t x_0$. For $\eta \geq 1$ it **diverges** (not shown).



Many variables: the gradient

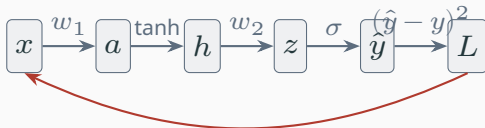
For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ the gradient is the vector of **partial derivatives**, $\nabla f = (\partial_{x_1} f, \dots, \partial_{x_n} f)$.



- ∇f points to the **steepest ascent** and is **perpendicular to the level curves**.
- Update: $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$.

The chain rule is backpropagation

$\hat{y} = \sigma(w_2 \tanh(w_1 x))$, $L = (\hat{y} - y)^2$: derivatives multiply along the path, from the loss back to each variable.



backward: multiply the local derivatives

- $\frac{\partial L}{\partial w_1} = 2(\hat{y} - y) \hat{y}(1 - \hat{y}) w_2 (1 - h^2) x$. Continue to x : $\partial L / \partial x$ (00-intro/02).
- JAX and Keras do this mechanically (*automatic differentiation*).

Logistic regression: loss and gradient

With $z_i = \mathbf{w} \cdot \mathbf{x}_i + b$:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_i \left[y_i \log \sigma(z_i) + (1 - y_i) \log (1 - \sigma(z_i)) \right]$$

$$\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}), \quad \nabla_{\mathbf{x}} J = (\sigma(z) - y) \mathbf{w}$$

- The first gradient trains the spam filter; the second says which **word** moves a score most: just the weight **w**. The loss is **convex**.

Numerical stability matters

Computing $\log(\text{sigmoid}(z))$ naively fails for confident predictions ($\sigma(-800)$ underflows to 0, and $\log 0 = -\infty$).

z	naive $\log \sigma(z)$	stable (<code>log_sigmoid</code>)
-800	$-\infty$	-800
-30	-30.000	-30.000
0	-0.693	-0.693

- Use $\log \sigma(z) = -\text{softplus}(-z)$ and $\log \sum_i e^{\ell_i} = m + \log \sum_i e^{\ell_i - m}$ (*log-sum-exp*).
- Same trick makes Naive Bayes work on long messages (notebooks/01-spam/01).

When there is no gradient

Use it when $f(\mathbf{x})$ is:

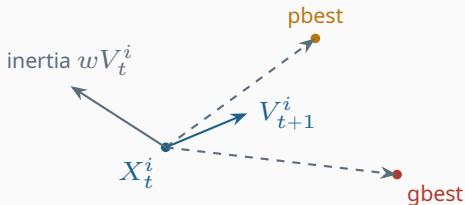
- **non-smooth** or discontinuous,
- **noisy**,
- **expensive** to evaluate,
- a **black box**: only inputs and outputs are visible (a scoring API).

Examples: genetic algorithms, differential evolution, simulated annealing, **particle swarm optimisation**.

Particle swarm optimisation (PSO)

Kennedy and Eberhart (Kennedy and Eberhart 1995): a swarm searches for food; each bird remembers its own best place and knows the best place of the flock.

$$V_{t+1}^i = w V_t^i + c_1 r_1 (\text{pbest}^i - X_t^i) + c_2 r_2 (\text{gbest} - X_t^i)$$
$$X_{t+1}^i = X_t^i + V_{t+1}^i$$



PSO in practice

- **Cost:** particles $\times$ iterations evaluations; each particle is independent, so it **parallelises** trivially.
- **Parameters:** inertia w (keep direction), c_1 (own memory), c_2 (swarm); they trade exploration for exploitation.
- Works on a non-convex “egg-carton”
 $f(x, y) = (x - 3.14)^2 + (y - 2.72)^2 + \sin(3x + 1.41) + \sin(4y - 1.73)$: PSO found $f = -1.8084$ at $(3.19, 3.13)$; the exhaustive grid optimum is -1.8083 (notebooks/00-intro/03).
- **No guarantee** of the global optimum: run several times.

Gradient descent versus PSO

Feature	Gradient descent	PSO
Needs	the gradient	function values only
Best for	smooth, convex problems	noisy, non-smooth, non-convex
Search	deterministic, one path	stochastic, a population
Local minima	can get stuck	can jump out
Cost	cheap per step, many steps	many evaluations per step
Analogy	one hiker downhill	a flock searching

Linear regression: four solvers, one answer

$$h(x) = mx + b, \text{ cost } e(m, b) = \frac{1}{2n} \sum_i (y_i - h(x_i))^2.$$

Same data (notebooks/00-intro/05):

Solver	Needs	RMSE
Closed form (lstsq)	linear model	6.930
Gradient descent	gradient	6.930
Newton	gradient + Hessian	6.930
PSO	only the loss	6.930

- Same problem, same answer; the solvers differ in **what they require** and what they **cost**.

Logistic regression for spam

- Features: TF-IDF vector of an SMS; model $\sigma(\mathbf{w} \cdot \mathbf{x} + b)$ with an L2 penalty.
- **Gradient descent** (300 steps) and **L-BFGS** reach the same loss and the same test score ($F1 \approx 0.94$, notebooks/01-spam/03); L-BFGS needs about a tenth of the iterations and no learning rate.
- **Choosing the threshold is optimisation too:** on Naive Bayes, a threshold of 0.9914 gives precision 0.992 and recall 0.930 (notebooks/01-spam/01).
- A **cost-sensitive** objective, e.g. $c_{FP} \cdot FP + c_{FN} \cdot FN$, picks the operating point from the analysts' capacity.

Attacking is optimisation

Same loss, different variable: maximise the loss over the **input** under a budget (Szegedy et al. 2014; Madry et al. 2018).

$$\max_{\|\delta\|_{\infty} \leq \varepsilon} \mathcal{L}(\theta; \mathbf{x} + \delta, y) \implies \delta \approx \varepsilon \operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}) \text{ (FGSM)}$$

Digits, $\varepsilon = 0.2$, 30 images (notebooks/00-intro/11):

Attack	Optimiser	Success	Cost
Random noise	none	0.07	none
FGSM	one gradient step	0.90	1 gradient
PSO (black-box)	derivative-free	0.97	~1,300 queries

Conclusion

- Optimisation is the engine of ML: a **model** plus a **loss** plus an **optimiser**.
- **Gradient-based** methods need $\nabla \mathcal{L}$ (automatic differentiation provides it); **derivative-free** methods need only evaluations.
- Choosing the solver is choosing **what you can observe** and **what a step costs**: exactly what separates a white-box from a black-box attacker.
- **Next**: apply it, on spam (Naive Bayes, logistic regression, fastText, embeddings), then break the detector.

Anderson, Hyrum S., Anant Kharkar, Bobby Filar, David Evans, and Phil Roth. 2018. *Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning*. arXiv:1801.08917.

Arp, Daniel, Erwin Quiring, Feargus Pendlebury, et al. 2022. "Dos and Don'ts of Machine Learning in Computer Security." *31st USENIX Security Symposium*.

Bibliography ii

Axelsson, Stefan. 2000. "The Base-Rate Fallacy and the Difficulty of Intrusion Detection." *ACM Transactions on Information and System Security* 3 (3).

Biggio, Battista, Blaine Nelson, and Pavel Laskov. 2012. "Poisoning Attacks Against Support Vector Machines." *29th International Conference on Machine Learning (ICML)*.

Deng, Gelei, Yi Liu, Víctor Mayoral-Vilches, et al. 2024. "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing." *USENIX Security Symposium*.

Bibliography iii

- Denning, Dorothy E. 1987. "An Intrusion-Detection Model." *IEEE Transactions on Software Engineering* SE-13 (2): 222-32.
- Fang, Richard, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. *LLM Agents Can Autonomously Exploit One-Day Vulnerabilities*. arXiv:2404.08144.
- Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.

Bibliography iv

Graham, Paul. 2002. *A Plan for Spam*.

<https://paulgraham.com/spam.html>.

Gu, Tianyu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. "BadNets: Evaluating Backdooring Attacks on Deep Neural Networks." *IEEE Access* 7.

Hitaj, Briland, Paolo Gasti, Giuseppe Ateniese, and Fernando Perez-Cruz. 2019. "PassGAN: A Deep Learning Approach for Password Guessing." *Applied Cryptography and Network Security (ACNS)*.

Kennedy, James, and Russell Eberhart. 1995. "Particle Swarm Optimization." *IEEE International Conference on Neural Networks*.

Lowd, Daniel, and Christopher Meek. 2005. "Good Word Attacks on Statistical Spam Filters." *Conference on Email and Anti-Spam (CEAS)*.

- Madry, Aleksander, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. "Towards Deep Learning Models Resistant to Adversarial Attacks." *International Conference on Learning Representations (ICLR)*.
- Papernot, Nicolas, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. "Practical Black-Box Attacks Against Machine Learning." *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*.

Sommer, Robin, and Vern Paxson. 2010. "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection." *IEEE Symposium on Security and Privacy*.

Souly, Alexandra, Javier Rando, Ed Chapman, Xander Davies, et al. 2025. *Poisoning Attacks on LLMs Require a Near-Constant Number of Poison Samples*.
arXiv:2510.07192.

Szegedy, Christian, Wojciech Zaremba, Ilya Sutskever, et al. 2014. "Intriguing Properties of Neural Networks." *International Conference on Learning Representations (ICLR)*.

Tramèr, Florian, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. "Stealing Machine Learning Models via Prediction APIs." *25th USENIX Security Symposium*.

Vassilev, Apostol, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. 2025. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST AI 100-2e2025. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.AI.100-2e2025>.

Zhang, Cen, Younggi Park, Fabian Fleischer, et al. 2026. *SoK: DARPA's AI Cyber Challenge (AIXCC): Competition Design, Architectures, and Lessons Learned*. arXiv:2602.07666.